

INITIATIVE OF THE EXCELLENCE - INVESTMENT FOR THE FUTURE



HARMFUL INSECT CLASSIFICATION AND ALERT SYSTEM FOR RICE FIELDS USING IMAGE PROCESSING

Final Report

Supervisor : Mr. Van Lic Tran

Submitted by: - Goettmann-Vogel Jean

- Bottero Adam

Abstract

The goal of the 'Harmful insect classification and alert system for rice fields using image processing' was to configure a device to, once set up inside a trap, detect and classify the harmful insect species and alert the user if one was detected. More specifically, the objectives were to build an IA model for the detection and classification, configure the microcontroller around this model for the inference and finally enable the communication between the user and the device through Wifi and an app.

After a first attempt using an image classification model in a patch detection system, we finally chose to use an object detection model.

At first, we built the model with the main priority being its size, as the used microcontroller has low memory and computational capabilities for object detection. Once what seemed to be the best model achievable, we configured the microcontroller around it, setting up the video capture, image processing for the model as well as outputting the results. Finally, we built the user app.

The model in use is a MobileNetV2 - Tiny YOLO Object Detection model, achieving a great overall accuracy, with scores for each of the 24 different classes going from 80% for the worst class up to 90% (AP50) for the best class. The final model size was just below 7.5Mo as a C header file. The inference time is a little bit over 15 seconds.

While we managed to build an object detection model that fits inside the microcontroller and can be inferred, the main flaw is the very high inaccuracy for several classes resulting in them not being detected at all by the device.

Acknowledgements

We would like to thank:

- The DNIIT and its Director, Dr. Lê Thành Nhân for granting us the opportunity of this internship, in this institution.
- Our supervisor, Mr. Van Lic Tran, who helped and accompanied us throughout this internship, giving us guidance.
- Võ Hoàng, for his advice on the project.
- Đặng Từ, who helped us settle in DaNang and with the everyday life in Vietnam.

Table of contents

Final Report	1
Abstract	2
Acknowledgements	3
Table of contents	4
Introduction	5
Material and Methods	6
Hardware	6
Software environment	6
Dataset	6
Model Architecture	8
Input.....	8
Backbone – MobileNetV2.....	8
Detection Head – YOLO-inspired	8
Output Structure.....	9
Post-processing (Inference)	9
Model Characteristics.....	9
Model training and results	10
MobilenetV2 - Tiny YOLO general characteristics	10
Overall Performance	11
Export and quantization	12
Deployment	13
Header Includes & Model Embedding	13
Initialization	14
Camera operation	14
AI Model Inference	14
Web server & API interaction	14
Memory and performance monitoring.....	15
Results	16
Model.....	16
Deployment.....	16
Discussions	16
Serial Monitoring	17

Introduction

Rice cultivation is very important in Vietnam as the country is among the world's top producers and exporters. The rice sector contributes significantly to the national economy, providing livelihoods for millions and generating substantial foreign revenue. The success of Vietnam's rice industry has a direct impact on national economic growth and stability.

Vietnam's tropical climate creates ideal conditions for the proliferation of various insect pests that damage rice crops. Infestations by pests can decrease rice yields, causing substantial economic losses for farmers and impacting national production targets. The cost of pest control measures, including pesticides, also adds to the economic burden.

Thus, finding better ways to fight insect pests causing damage to rice crops is a necessity, as it can help to reduce the losses and achieve a greener and more sustainable agriculture by reducing the use of chemical pesticides. In our case, this project aims at gathering meaningful data about pest population in rice fields in order to adapt the response accordingly.

The main constraint in this project being the memory and computational capabilities of the microcontroller, the most important target was the size and computational complexity of the model. The accuracy was then to be brought as high as possible within this context. The inference time was not a priority as pest detection doesn't require real time but could even be done with a daily report.

Material and Methods

Hardware

ESP32-S3 Dev Module: It is a low-power, feature-rich development board designed for Internet of Things (IoT) applications, particularly those requiring image capture and processing. The ESP32-S3 provides robust wireless communication, including both Wi-Fi and Bluetooth 5.0 and includes a 16 Mb flash memory as well as a 8Mb PSRAM (Parallel Synchronous Random-Access Memory).

OV2640 Camera: It is a 2-megapixel (MP) CMOS image sensor, widely used in small embedded systems and IoT devices due to its low power consumption, small size, and ease of use. It's often paired with microcontrollers like the ESP32. The maximum resolution is 1600x1200, but it can also output lower resolution at higher frame rates.

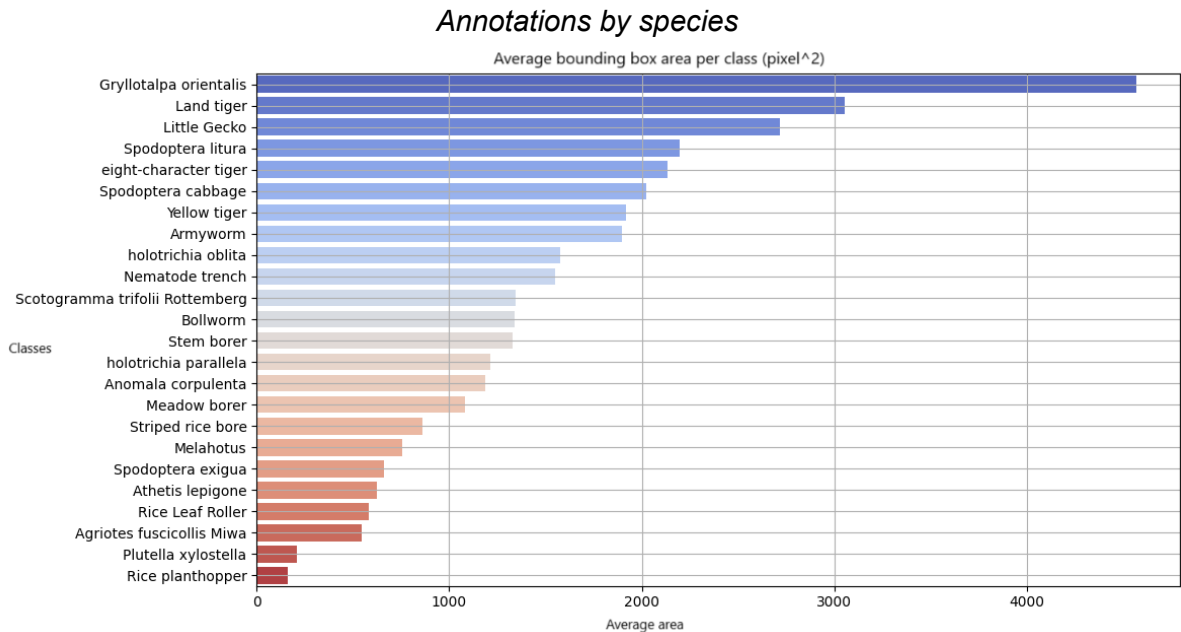
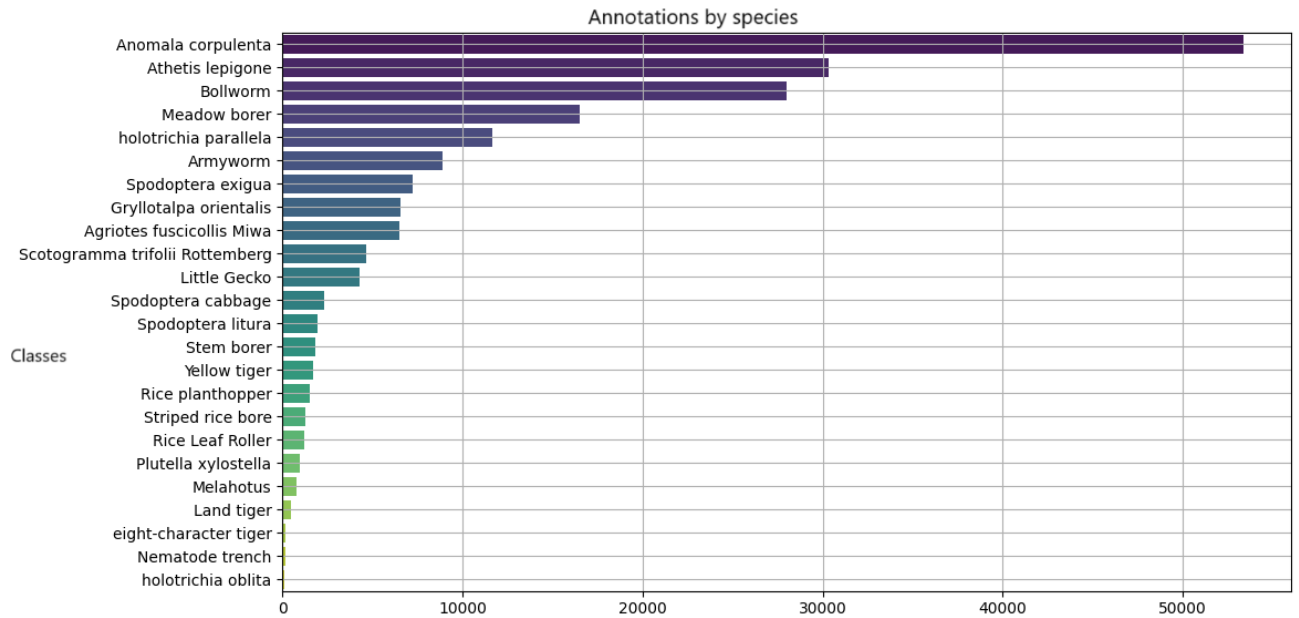
Software environment

WSL2: Model training was done in a WSL2 (Windows Subsystem for Linux 2) environment, which is a Windows feature allowing the user to run a genuine Linux environment directly on your Windows machine without the need for a traditional virtual machine or dual booting. Details about the virtual environments and the requirements can be found on GitHub.

Arduino IDE 2.0: This was the IDE (Integrated Development Environment) used to write, compile, and upload sketches and codes to the ESP32S3 Dev Module.

Dataset

Pest24: This dataset, available on Kaggle in the Pascal VOC XML format, was created by researchers from the Institute of Intelligent Machinery of the Source of Material Science Research of the Chinese Academy of Sciences in Hefei. It features around 30.000 field pest annotated images in high resolution (2095 by 1944 pixels), with dense distribution of ultra small objects (insects) in each image.



Average bounding box area per class

The two graphs above clearly show the main flaws of this dataset:

- The dataset is heavily unbalanced with some species being featured several thousands of times while some others barely appear a hundred times.
- The average size of insects varies widely between the different classes.

The results shown later in this report prove that the biggest difficulty was the data imbalance, as the least present pests have the worst detection accuracy (some of them with 0% accuracy). Data augmentation could only partially fix this problem as it consisted in creating variations of already existing images to artificially boost a species' presence. However, since the images feature a dense group of insects, this process increases the presence of all the other species in the image, sometimes even worsening the unbalance.

The insect's size, on the other hand, didn't happen to be a big problem as the various models managed to get acceptable scores on extra-small insects as long as there were enough images for this specific species in the dataset.

Model Architecture

Our insect detector model combines a lightweight feature extractor with a single-shot detection head, optimized for use on the ESP32-S3 as it is very lightweight and requires light computational power.

Input

- **Image size:** 224×224 pixels
- **Channels:** RGB (3 channels)
- **Preprocessing:** Images are resized to the input size; no heavy augmentations applied during inference.

Backbone – MobileNetV2

- **Purpose:** Extract rich spatial and semantic features from the input image.
- **Width multiplier (α):** $1.0 \rightarrow$ full capacity MobileNetV2.
- **Pretrained on:** ImageNet for improved feature initialization.
- **Output feature map:**
 - Taken from layer **block_13_expand_relu**.
 - Spatial resolution: 14×14 (stride = 16).
 - Chosen to balance detection precision and computation speed.

Detection Head – YOLO-inspired

- **Goal:** Convert backbone features into bounding box and class predictions.
- **Structure:**
 - **1×1 Convolution:** Adjusts channel depth to 256 without spatial change.
 - **Two Depthwise Separable 3×3 Convolutions:**
 - Maintain spatial resolution (14×14).
 - Add non-linear feature transformations while keeping computations light.
 - **Batch Normalization + ReLU6:** After each convolution for stability and efficiency.
 - **Final 1×1 Convolution:** Outputs detection predictions.

Output Structure

- **Grid size:** 14×14 (one prediction cell per grid location).
- **Per-anchor output:**
- **1 objectness score** (probability of object presence).
- **4 box offsets:**
- tx, ty $\rightarrow$ offsets from cell center (normalized).
- tw, th $\rightarrow$ log-scale adjustments for width and height.
- **24 class probabilities** (softmax applied per anchor).
- **Total per cell:** $1 + 4 + 24 = 29$ values.
- **Overall output tensor shape:** (14, 14, 3, 29).

Post-processing (Inference)

- **Decoding:**
- Convert predicted offsets into absolute normalized box coordinates.
- Multiply objectness with class probabilities for final confidence scores.
- **Filtering:**
- Confidence thresholding to remove low-score detections.
- Non-Maximum Suppression (NMS) to eliminate overlapping boxes.
- **Final result:** List of bounding boxes, each with a class label and confidence score.

Model Characteristics

- **Total parameters:** ~924,823
- **Model size (float32):** ~3.53 MB
- **Quantized size (INT8):** ~1.14 MB $\rightarrow$ suitable for microcontrollers like ESP32-S3.
- **Single-scale detection:** Optimized for speed and reduced complexity.

Model training and results

MobilenetV2 - Tiny YOLO general characteristics	
Attribute	Value
Model Name	MDET_Fixed_MobileNetV2_YOLO
Architecture Backbone	MobileNetV2
Detection Head	YOLO
Input Shape	224 × 224 × 3
Output Shape	14 × 14 × 3 × 29
Parameters	924,823
Model Size (Float)	3.53 MB
Original Size Before Quantization	11.09 MB
Quantized Size (INT8)	1.14 MB
Compression Ratio	89.7 %
Quantization Quality	EXCELLENT
Epochs	30
Early Stopping	Yes
Best Validation Loss	0.07701
Final Train Loss	0.0258
Final Validation Loss	0.0833
Dataset Size	25,378 images / 192,424 annotations
Classes	24
Train/Validation Split	80 % / 20 %
Imbalance Ratio	494.0
TFLite Export	Yes
INT8 Quantization	Yes

C Header Generated	Yes
Deployment Compatibility	TFLite Micro

General model characteristics

Overall Performance	
Metric	Value
mAP@0.50	0.9006
Overall Precision	0.6375
Overall Recall	0.2732
Overall F1-score	0.3825
Validation Samples	5,076
Total GT Annotations	37,456
Total Detections	16,051
True Positives	10,233
False Positives	5,818
Confidence Threshold	0.25
IoU Threshold	0.45
Eval Image Size	224

Model general performance

Although the mAP50 value is very high (90%) it is mitigated by the poor overall recall score (27.3%). To explain this in simpler terms, the model struggles to localize insects with confidence in an image, but when it does so, the predicted class is the right one with 90% chances.

AP per class					
Class Name	AP@0.50	GT Count	Class Name	AP@0.50	GT Count
holotrichia oblita	0.9591	18	Spodoptera cabbage	0.9059	486

Nematode trench	0.9545	26	Little Gecko	0.8993	901
eight-character tiger	0.9530	30	Scotogramma trifolii Rottemberg	0.8993	905
Land tiger	0.9267	105	Gryllotalpa orientalis	0.8934	1,246
Melahotus	0.9232	138	Spodoptera exigua	0.8898	1,498
Plutella xylostella	0.9172	210	Agriotes fuscicollis Miwa	0.8893	1,519
Rice planthopper	0.9133	264	Armyworm	0.8868	1,759
Rice Leaf Roller	0.9131	278	holotrichia parallela	0.8798	2,365
Striped rice bore	0.9126	305	Meadow borer	0.8712	3,224
Yellow tiger	0.9114	331	Bollworm	0.8483	5,558
Spodoptera litura	0.9107	370	Athetis lepigone	0.8455	5,918
Stem borer	0.9107	366	Anomala corpulenta	0.8000	9,636

Model's AP per class

The GT-count reflects the number of actual, manually annotated instances of a class in the validation dataset.

Export and quantization

For the model to be usable on an ESP32 Microcontroller, it needs to be exported as a TensorFlow Lite file, which is the microcontroller-oriented format within the TensorFlow framework. The process transforms a full precision Keras model (stored in H5 format) into an optimized model suitable for embedded deployment.

The first step is to create a representative dataset, in our case featuring 100 synthetic 224x224x3 RGB images to calibrate the quantization process. During the quantization, the 32-bit-floating point weights and activations are converted to 8-bit integers while maintaining model accuracy through careful calibration using the representative dataset. The model is finally converted as a C header file, making it directly embeddable in ESP32-S3 firmware.

The quantization of the model is used to dramatically improve memory efficiency as it reduces the model's size by approximately 75% compared to the original FP32 precision format, allowing the model to fit the very limited ESP32 memory. Additionally, quantization significantly enhances computational speed since 8-bit integer operations are way faster. Ultimately, power

consumption is also lowered due to reduced memory access and simpler arithmetic operations. In our case, quantization was mostly used to comply with the memory limitations as our application didn't require real-time nor had any power consumption limitations.

Deployment

As mentioned in the 'Software Environment' section, the model was deployed using the Arduino IDE (2.6).

Header Includes & Model Embedding

The working directory features:

- The **Arduino sketch** which is the main file (.ino).
- The **model** as C header (.h)
- The **list of classes** as a C header (h)

Note: As the Arduino environment is C-based, the model cannot be used as a TensorFlow Lite file and needs to be converted as a C header which greatly increases the file's size.

Additionally, the program requires the following libraries:

- Arduino.h: Base library required for all Arduino programs.

Networking Libraries:

- Wifi.h: Used to connect the ESP32 to a Wi-Fi network.
- Webserver.h : Used to handle simple HTTP requests.
- WifiManager.h: Auto-configuration library that helps connect to Wi-Fi networks.

TensorFlowLite Micro:

- Chirale_TensorFlowLite.h: Wrapper that includes TFLite headers and handles Arduino integration.
- tensorflow/lite/micro/all_ops_resolver.h: Registers all TFLite Micro Operators.
- tensorflow/lite/micro/micro_interpreter.h: Main TFLite Micro class used to run inference on microcontrollers. Handles tensor allocation, model invocation, etc.
- tensorflow/lite/schema/schema_generated.h: Loads and interprets the structure of the model.
-

ESP hardware specific libraries:

- esp_camera.h: Driver for the OV2640 camera, used to initialize the camera, capture images etc.
- esp_timer.h: Allows precise timing operations like measuring inference time in microseconds or milliseconds.
- esp_heap_caps.h: Gives you access to ESP32's memory management. Used for performance optimization or debugging memory leaks.

Initialization

This first step prepares the device for operation.

- 1- The OV2640 camera gets initialized and configured for image capture. The image format and size are specified. In our case JPEG to decrease the memory use and QVGA (320x240) which is the closest size to our model's input size. Some more parameters are specified such as frame management and pins configuration.
- 2- The TensorFlow Lite interpreter gets initialized. This component is responsible for the model's execution, enabling efficient and optimized execution on the microcontroller. The model is loaded, and the tensors (multi-dimensional data containers) are allocated during this step.
- 3- The Wi-Fi connection is established using the WiFiManager. For easier use, the Wi-Fi credentials are not hard coded in the sketch. The user selects his hot spot on an interface and enters the credentials.
- 4- The HTTP routes are configured, and the API handlers are set up.
- 5- Finally, the web server is started.

Camera operation

Since the camera has already been initialized, the camera captures a frame buffer according to the defined configuration. It is stored in the 8-Mo PSRAM (Pseudo-Static Random Access Memory).

AI Model Inference

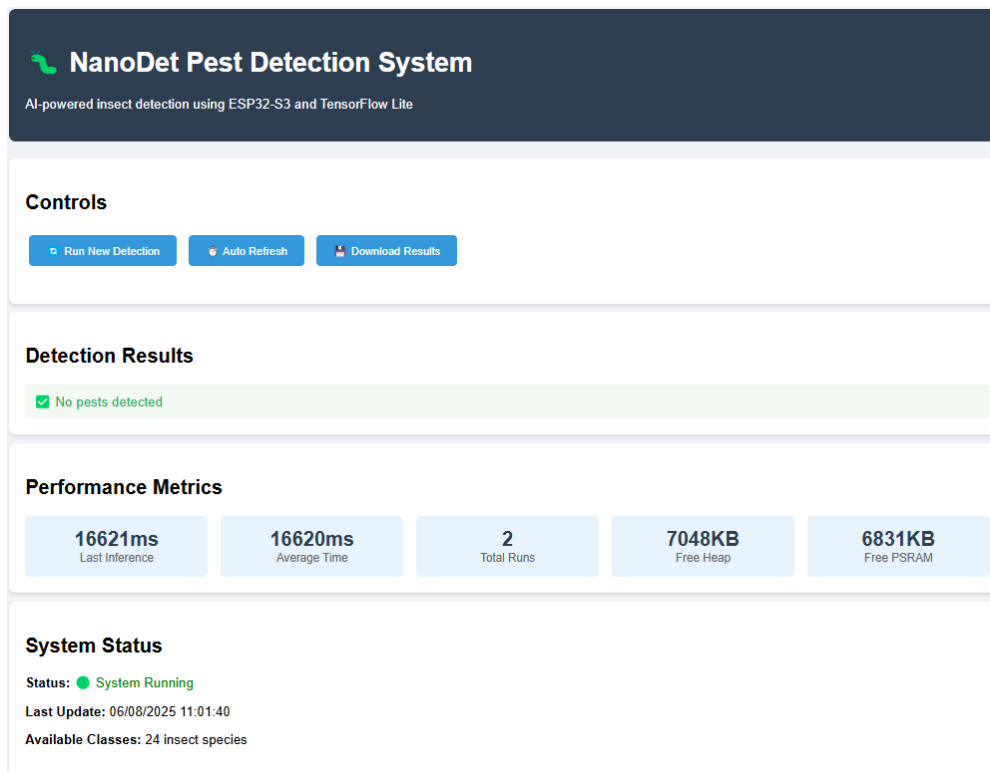
Before the model's inference, the image needs to be preprocessed. It gets resized to the model's input size (288x288) and converted to RGB888. Although the camera can capture images in a RGB format instead of JPEG, this heavily increases memory usage for image capture. Only then is the image's data copied into the TFLite input tensor. The model is invoked and delivers and outputs predictions that are then post-processed. The thresholds are applied to define whether the confidence level is high enough to consider the prediction accurate and the final results are stored in a list of detections.

The program also features a Non-Maximum Suppression (NMS) function. Its purpose is to filter overlapping boxes to reduce duplicates. The idea is to remove boxes that overlap above a specific threshold.

Web server & API interaction

The results (detected pests, classes, confidence and bounding box) are stored in a JSON string, which is then sent to the server for the user to visualize in the interface.

Additionally, at any time the server is waiting for a request from the client as the user can ask for a new model's inference, switch to auto refresh mode or download the results. The performance metrics are permanently displayed, as well as the system's status.



Pest Detection System Web Browser view

Memory and performance monitoring

Since this is only a prototype, memory and performance monitoring are included in the loop, constantly tracking the memory used during the process to detect any issues. Additionally, the inference time is measured. After the whole inference and web communication process, the memory is freed of the now useless data, although the images are heavily reduced for inference (288x288 pixels).

Results

The goal was to train a model able to detect and classify harmful insects trapped in a rice field, an embed it into an ESP32 micro-controller to help farmers fight pests more efficiently and accurately. A user app to monitor the detection was also included.

- Model

The hardest part about the model was making it light enough to fit in the very limited EPS32 memory. The final model we trained does fit, being just below 7.5 Mo as a C header. The results are very encouraging, especially with the exceptionally high AP50 score. There is obviously room for improvement as the recall score remains very low. This flaw can be reduced by lowering the confidence threshold. Overall, the model should be very able to detect if any harmful species featured in the dataset is present in the rice field but will also underestimate the total population for each class.

- Deployment

The model is deployed on the ESP32-S3 Dev Module. The memory management has been optimized to avoid overflow during the whole process as shown by Serial Monitor debug prints. The inference time is above 15 seconds, which we consider to be acceptable looking at the context and task.

Discussions

This project successfully delivered a working prototype for harmful insect detection on rice plantations, with the final model deployed on an ESP32-S3 and able to communicate predictions via Wi-Fi. The development process, however, faced several challenges.

Model training was the most time-consuming stage: the initial “classification by patch” approach was abandoned in favor of object detection, which proved more suitable but raised new constraints. The main difficulty was finding an object detection architecture that met the strict memory and computational limits of the ESP32-S3, as model size could not be reliably predicted prior to training.

The dataset’s severe class imbalance further complicated the task, limiting detection accuracy for under-represented species despite good results on the majority classes.

Deployment also presented hurdles: the Arduino TFLite Micro library was unable to handle certain model operations (notably with NanoDet models), forcing architectural compromises; image preprocessing was challenging to implement efficiently given the limited memory; and the user interface remains rudimentary, serving primarily for testing rather than field use. Future work should focus on finding a way to compensate for the dataset imbalance, refining the model architecture to improve recall without exceeding hardware limits, optimizing preprocessing for speed and stability, and developing a more user-friendly interface for practical deployment.

Serial Monitoring

The expected behavior of the program is shown below using screenshots from the Arduino IDE's Serial Monitor. These debug logs are not accessible through the web interface/user app.

```
00:27:39.927 -> [INFO] WiFi configuration saved
00:27:40.934 -> *wm:config portal exiting
00:27:40.934 -> [INFO] WiFi connected successfully
00:27:40.934 -> [INFO] IP address: 172.21.0.51
00:27:40.934 -> [INFO] Signal strength: -69 dBm
```

WiFi connexion is successful

```
00:27:40.934 -> [INFO] === Pre-Camera Memory Analysis ===
00:27:40.934 -> [INFO] Total DRAM: 332332 bytes (324 KB)
00:27:40.934 -> [INFO] Free DRAM: 246784 bytes (241 KB)
00:27:40.934 -> [INFO] Largest DRAM block: 196596 bytes (191 KB)
00:27:40.934 -> [INFO] Free PSRAM: 8174 KB
00:27:40.979 -> [INFO] Initializing camera first to secure DRAM...
00:27:40.979 -> [INFO] Testing DRAM allocation for camera task stack...
00:27:40.979 -> [INFO] ✓ Camera task stack test allocation successful
00:27:41.114 -> [INFO] Camera initialized successfully
00:27:41.114 -> [INFO] Post-camera DRAM: 225856 bytes (220 KB)
00:27:41.150 -> [INFO] Post-camera largest block: 180212 bytes (175 KB)
00:27:41.150 -> [INFO] Testing camera frame capture...
00:27:41.196 -> [INFO] ✓ Camera frame capture successful
00:27:41.196 -> [INFO] Frame size: 320x240, length: 5578
00:27:42.190 -> [INFO] Camera stabilization complete
```

Camera configuration and memory allocation

```
00:27:42.190 -> [INFO] === Pre-TensorFlow Memory Analysis ===
00:27:42.190 -> [INFO] Pre-TF Free DRAM: 225792 bytes (220 KB)
00:27:42.190 -> [INFO] Pre-TF Largest DRAM block: 8257524 bytes (8063 KB)
00:27:42.190 -> [INFO] Allocating tensor arena (1536 KB in PSRAM)...
00:27:42.190 -> [INFO] Available PSRAM before allocation: 8143 KB
00:27:42.190 -> [INFO] Trying to allocate 1536 KB tensor arena...
00:27:42.190 -> [INFO] ✓ Tensor arena allocated successfully: 1536 KB
00:27:42.223 -> [INFO] Post-allocation Free DRAM: 225996 bytes (220 KB)
00:27:42.223 -> [INFO] Post-allocation Largest DRAM block: 6684660 bytes (6527 KB)
00:27:42.223 -> [INFO] DRAM change: 204 bytes
00:27:42.223 -> [INFO] Remaining PSRAM: 6575 KB
```

Tensor arena allocation

```

00:27:42.223 -> [INFO] === TensorFlow Initialization ===
00:27:42.223 -> [INFO] Initializing TensorFlow Lite model...
00:27:42.223 -> [INFO] Model loaded successfully
00:27:42.223 -> [INFO] DRAM after model load: 225996 bytes (change: 0)
00:27:42.263 -> [INFO] Creating interpreter...
00:27:42.263 -> [INFO] DRAM after interpreter creation: 225780 bytes (change: -216)
00:27:42.263 -> [INFO] Allocating tensors...
00:27:42.295 -> [INFO] 🔍 TENSOR ARENA ANALYSIS:
00:27:42.295 -> [INFO] Arena allocated: 1536 KB
00:27:42.295 -> [INFO] Arena actually used: 1252 KB
00:27:42.295 -> [INFO] Arena utilization: 81.5%
00:27:42.328 -> [INFO] Free arena space: 283 KB
00:27:42.328 -> [INFO] ✅ Arena size appears adequate
00:27:42.328 -> [INFO] DRAM after tensor allocation: 225828 bytes (change: 48)
00:27:42.328 -> [INFO] Total DRAM consumed by TensorFlow: 168 bytes

```

Interpreter creation and Tensor Arena check

```

00:27:42.328 -> [INFO] 🔍 FULL INT8 MODEL QUANTIZATION PARAMETERS:
00:27:42.328 -> [INFO] Input scale: 0.003922
00:27:42.328 -> [INFO] Input zero point: -128
00:27:42.328 -> [INFO] Class output scale: 0.003906
00:27:42.361 -> [INFO] Class output zero point: -128
00:27:42.361 -> [INFO] BBox output scale: 0.008074
00:27:42.361 -> [INFO] BBox output zero point: 82
00:27:42.361 -> [INFO] Input type: 9
00:27:42.361 -> [INFO] Class output type: 9
00:27:42.361 -> [INFO] BBox output type: 9
00:27:42.361 -> [INFO] Input tensor: 150528 bytes
00:27:42.361 -> [INFO] Class output tensor: 24 bytes
00:27:42.361 -> [INFO] BBox output tensor: 4 bytes
00:27:42.361 -> [INFO] Input dimensions: [1,224,224,3]
00:27:42.361 -> [INFO] Class output dimensions: [1,24,-196342]
00:27:42.394 -> [INFO] BBox output dimensions: [1,4,-196070]
00:27:42.394 -> [INFO] Free heap: 6958720 bytes
00:27:42.394 -> [INFO] Free PSRAM: 6732892 bytes

```

Quantization parameters check

```

00:27:42.394 -> [INFO] === Post-TensorFlow Camera Test ===
00:27:42.394 -> [INFO] Final DRAM state:
00:27:42.394 -> [INFO] Free DRAM: 225828 bytes (220 KB)
00:27:42.394 -> [INFO] Largest DRAM block: 6684660 bytes (6527 KB)
00:27:42.394 -> [INFO] Testing camera after TensorFlow initialization...
00:27:42.394 -> [INFO] ✅ POST-TENSORFLOW camera test successful!

```

Post-TensorFlow camera test

```

00:27:42.428 -> [INFO]   Frame: 320x240
00:27:42.428 -> [INFO]   Setting up HTTP server...
00:27:42.428 -> [INFO]   HTTP server started on port 80
00:27:42.428 -> [INFO]   === System Ready ===
00:27:42.428 -> [INFO]   Available classes: 24
00:27:42.428 -> [INFO]   CPU frequency: 240 MHz
00:27:42.428 -> [INFO]   XTAL frequency: 40 MHz
00:27:42.428 -> [INFO]   APB frequency: 80 MHz
00:27:42.428 -> [INFO]   Free heap: 6957460 bytes
00:27:42.428 -> [INFO]   Free PSRAM: 6732704 bytes
00:27:42.428 -> [INFO]   Access web interface at: http://172.21.0.51

```

Server and system check

```

00:27:42.462 -> [INFO]   Processing real camera image
00:27:42.544 -> [INFO]    INPUT VALIDATION:
00:27:42.544 -> [INFO]   Sample input values: [-128,14,-54]
00:27:42.544 -> [INFO]   Input range check: Expected INT8 (-128 to 127), Scale=0.003922, ZP=-128
00:27:42.544 -> [INFO]   Input value range (first 1000): [-128, 126]
00:27:42.585 -> [INFO]   Zero values: 5/1000 (0.5%)
00:27:42.585 -> [INFO]    Running inference with real camera image...
00:27:42.585 -> [INFO]   Interpreter state check:
00:27:42.585 -> [INFO]   - Input tensor valid: YES
00:27:42.585 -> [INFO]   - Output tensor valid: YES
00:27:42.585 -> [INFO]   - Input tensor data: 1008983440
00:27:42.585 -> [INFO]   - Output tensor data: 1008983440
00:27:46.830 -> [INFO]   Inference completed successfully with status: 0
00:27:46.830 -> [INFO]   Memory after inference - Free heap: 224756 bytes, Free PSRAM: 6732704 bytes

```

Input validation and inference

```

00:27:46.898 -> [INFO]    OUTPUT TENSOR ANALYSIS:
00:27:46.898 -> [INFO]   Class output dimensions: [1,24]
00:27:46.898 -> [INFO]   BBox output dimensions: [1,4]
00:27:46.898 -> [INFO]   Number of classes: 24
00:27:46.898 -> [INFO]   Class output scale: 0.003906, ZP: -128
00:27:46.930 -> [INFO]   BBox output scale: 0.008074, ZP: 82
00:27:46.930 -> [INFO]   Sample class outputs: [-115,-115,-118,-123,-125,-121]
00:27:46.930 -> [INFO]   Sample bbox outputs: [-46,74,81,48]
00:27:46.930 -> [INFO]   Zero point analysis: 0/24 values are zero point (0.0%)
00:27:46.930 -> [INFO]   Best class: 13 with score: 0.164
00:27:46.930 -> [INFO]   Bounding box: center=(-1.03,-0.06), size=(-0.01x-0.27)
00:27:46.930 -> [INFO]    DETECTION RESULTS:
00:27:46.930 -> [INFO]   Detections found: 1
00:27:46.963 -> [INFO]   Total classes processed: 24
00:27:46.963 -> [INFO]   Inference completed in 4509ms

```

Output tensor analysis and early metrics

```

00:27:46.963 -> =====
00:27:46.963 -> 🐛 NANODET PEST DETECTION RESULTS
00:27:46.963 -> =====
00:27:46.963 -> ✅ PESTS DETECTED:
00:27:46.963 -> -----
00:27:46.963 -> Detection #1:
00:27:46.963 -> 🐛 Insect: Rice planthopper (ID: 13)
00:27:46.963 -> 📊 Confidence: 16.4%
00:27:46.963 -> 📏 Bounding Box: (-1.03,0.07) to (-1.04,-0.20)
00:27:46.997 -> -----
00:27:46.997 -> ⚡ PERFORMANCE METRICS:
00:27:46.997 -> 🕒 Inference Time: 4509 ms
00:27:46.997 -> 📧 Average Time: 4509.0 ms
00:27:46.997 -> 📊 Total Inferences: 1
00:27:46.997 -> 💾 Free Heap: 6794 KB
00:27:46.997 -> 🧠 Free PSRAM: 6574 KB
00:27:46.997 -> =====

```

Final predictions and metrics

